

Sistema de Recomendación Paralelo para E-Commerce

Ana María Ramírez, María José Miranda, Siloé Campos, Julio Gutiérrez, David Mora

Computación Paralela y Distribuida

Prof. Johansell Villalobos Cubillo

LEAD University

San José, Costa Rica

Julio 2026

Resumen—El crecimiento del comercio electrónico ha incrementado de forma significativa el volumen de datos generados por las interacciones entre usuarios y productos. Estas interacciones, tales como visualizaciones, adiciones al carrito y compras, representan una fuente valiosa para construir sistemas de recomendación personalizados. Sin embargo, el procesamiento eficiente de millones de eventos requiere estrategias que superen las limitaciones de los enfoques secuenciales tradicionales. Este proyecto propone el diseño de un sistema de recomendación para e-commerce basado en procesamiento paralelo y distribuido, utilizando herramientas como Polars, Dask, RAPIDS cuML, PyTorch y Dash. La solución busca evaluar no solo la calidad de las recomendaciones, sino también el rendimiento computacional del pipeline mediante métricas como tiempo de ejecución, speedup, eficiencia y latencia.

Palabras clave—sistemas de recomendación, comercio electrónico, computación paralela, procesamiento distribuido, GPU, Polars, Dask, PyTorch.

I. INTRODUCCIÓN

El crecimiento acelerado del comercio electrónico ha incrementado de manera significativa el volumen de datos generados por las interacciones entre usuarios y productos. Cada visualización, adición al carrito y compra constituye una fuente de información valiosa que permite comprender las preferencias de los clientes y ofrecer experiencias de compra más personalizadas. Sin embargo, el procesamiento de millones de estos registros representa un desafío importante en términos de almacenamiento, análisis y capacidad de respuesta de los sistemas tradicionales, lo que hace necesario recurrir a técnicas de computación paralela y procesamiento distribuido.

En este contexto, los sistemas de recomendación se han convertido en un componente imprescindible de las plataformas de comercio electrónico, ya que permiten sugerir productos relevantes para cada usuario a partir de su comportamiento histórico y de los patrones observados en sus interacciones. Estas recomendaciones contribuyen a mejorar la experiencia del usuario, incrementan las tasas de conversión y fortalecen la fidelización de los clientes. No obstante, para que estos sistemas sean efectivos deben ser capaces de procesar grandes volúmenes de datos de manera eficiente y generar recomendaciones en tiempos de respuesta reducidos, incluso ante picos de tráfico como los que ocurren en eventos masivos de ventas.

Frente a esta necesidad, el presente proyecto propone el diseño, la implementación y la evaluación de un sistema de

recomendación personalizado para comercio electrónico, cuya optimización se logra mediante el aprovechamiento conjunto de paralelismo multinúcleo, aceleración por GPU y procesamiento distribuido. El objetivo es comprender cómo conceptos fundamentales de la computación paralela, como el speedup, la eficiencia, la sincronización y el profiling, se traducen en aplicaciones prácticas que impactan la vida diaria de millones de personas.

Como fuente de datos se utilizará el dataset RetailRocket E-commerce Dataset, el cual contiene aproximadamente 2.7 millones de eventos de interacción entre usuarios y productos, incluyendo visualizaciones, adiciones al carrito y transacciones, además de información sobre categorías y propiedades de los productos. La magnitud de este conjunto de datos ofrece un escenario adecuado para evaluar el impacto de distintas estrategias de paralelización tanto en tareas de análisis exploratorio y estadístico como en procesos de aprendizaje automático, permitiendo medir los beneficios de la computación paralela y distribuida aplicada a un problema real de alto impacto.

II. TRABAJO RELACIONADO

Los sistemas de recomendación se han convertido en una parte esencial de las plataformas digitales modernas, especialmente en el comercio electrónico. Su función principal es ayudar al usuario a encontrar productos relevantes dentro de catálogos cada vez más amplios. En este tipo de entornos, cada visualización, búsqueda, adición al carrito o compra deja una señal de comportamiento que puede utilizarse para comprender mejor los intereses de los clientes y generar recomendaciones personalizadas.

A lo largo del tiempo se han desarrollado distintos enfoques para construir sistemas de recomendación. Uno de los más utilizados es el filtrado colaborativo, el cual se basa en la relación entre usuarios y productos. La idea central es que, si varios usuarios han mostrado comportamientos similares en el pasado, es probable que compartan intereses hacia ciertos productos. Este enfoque resulta especialmente útil cuando se cuenta con grandes cantidades de interacciones históricas, como ocurre en plataformas de comercio electrónico.

Dentro del filtrado colaborativo, una de las técnicas más conocidas es la factorización matricial. Este método representa a usuarios y productos mediante vectores de características

latentes, lo que permite estimar afinidades entre ambos. En lugar de trabajar directamente con una matriz muy grande y dispersa de interacciones, el sistema aprende representaciones más compactas que facilitan el cálculo de recomendaciones. Esta idea ha sido ampliamente aplicada en escenarios donde se busca predecir preferencias a partir de datos históricos.

En el comercio electrónico, muchas veces no se cuenta con calificaciones explícitas de los usuarios. Es decir, los clientes no siempre indican si un producto les gustó o no mediante una puntuación directa. En su lugar, el sistema debe interpretar señales indirectas, como visitar un producto, agregarlo al carrito o realizar una compra. A este tipo de información se le conoce como retroalimentación implícita. Trabajar con esta clase de datos requiere un tratamiento cuidadoso, ya que no todas las acciones tienen el mismo nivel de importancia. Por ejemplo, una compra suele representar una señal más fuerte que una simple visualización.

Además de los métodos tradicionales, en años recientes han ganado relevancia los modelos basados en aprendizaje profundo. Estos enfoques permiten representar usuarios y productos mediante embeddings, es decir, vectores aprendidos automáticamente durante el entrenamiento. Arquitecturas como los modelos Two-Tower resultan útiles porque separan la representación del usuario y del producto, permitiendo comparar ambos espacios de manera eficiente durante la etapa de recomendación. Este tipo de modelo puede ser más flexible que los enfoques tradicionales, aunque también exige mayor capacidad computacional.

El reto de los sistemas de recomendación no se limita a lograr buenas métricas de predicción. En plataformas reales, también es necesario procesar grandes volúmenes de datos en tiempos razonables. Un sistema puede ser preciso, pero si tarda demasiado en generar recomendaciones, pierde utilidad práctica. Por esta razón, el rendimiento computacional es un aspecto fundamental. En escenarios de alto tráfico, como campañas masivas de ventas o eventos tipo Black Friday, el sistema debe responder rápidamente aunque aumente de forma significativa el número de usuarios e interacciones.

En este contexto, herramientas modernas de procesamiento de datos como Polars y Dask ofrecen ventajas importantes. Polars permite ejecutar operaciones tabulares de forma eficiente aprovechando múltiples núcleos de CPU y técnicas de optimización de consultas. Dask, por su parte, permite dividir el trabajo en particiones y distribuir tareas entre varios workers, lo que facilita el procesamiento de datasets grandes. Estas herramientas son relevantes para el proyecto porque permiten comparar enfoques tradicionales contra alternativas paralelas o distribuidas.

También existen herramientas orientadas a la aceleración por GPU, como RAPIDS cuDF y cuML. Estas librerías permiten ejecutar operaciones de análisis de datos y algoritmos de Machine Learning sobre GPU, lo que puede reducir considerablemente los tiempos de cómputo en tareas altamente paralelizables. En un sistema de recomendación, esta aceleración puede ser útil para procesos como clustering de usuarios, cálculo de similitudes, entrenamiento de modelos o generación

de recomendaciones en tiempo reducido.

A partir de estos antecedentes, el presente proyecto se ubica en la intersección entre sistemas de recomendación, análisis de datos a gran escala y computación paralela/distribuida. La propuesta no se enfoca únicamente en construir un modelo de recomendación, sino también en evaluar cómo distintas estrategias computacionales afectan el rendimiento del sistema. De esta manera, el proyecto busca demostrar que la calidad de las recomendaciones y la eficiencia del procesamiento deben analizarse de forma conjunta cuando se trabaja con datos reales de comercio electrónico.

III. MARCO TEÓRICO

III-A. *Sistemas de recomendación*

Los sistemas de recomendación son herramientas computacionales diseñadas para sugerir elementos relevantes a un usuario a partir de sus preferencias, historial de interacción o patrones de comportamiento. En el contexto del comercio electrónico, estos sistemas permiten recomendar productos que podrían ser de interés para cada cliente, facilitando la búsqueda dentro de catálogos extensos y mejorando la experiencia de compra.

En plataformas de e-commerce, las recomendaciones pueden generarse a partir de distintos tipos de información, como productos vistos, artículos agregados al carrito, compras realizadas o similitudes entre usuarios. Por esta razón, los sistemas de recomendación se consideran una aplicación importante del análisis de datos y del aprendizaje automático.

III-B. *Retroalimentación implícita en e-commerce*

En muchos sistemas de comercio electrónico no se cuenta con calificaciones directas de los usuarios. Es decir, los clientes no siempre indican explícitamente si un producto les gustó o no. En su lugar, el sistema registra acciones como visualizaciones, adiciones al carrito y compras. A este tipo de información se le conoce como retroalimentación implícita.

La retroalimentación implícita debe interpretarse cuidadosamente, ya que cada evento representa un nivel distinto de interés. Una visualización puede indicar curiosidad, una adición al carrito puede reflejar intención de compra y una transacción puede considerarse una señal fuerte de preferencia. En este proyecto, estos eventos se utilizan como base para construir relaciones entre usuarios y productos.

III-C. *Filtrado colaborativo*

El filtrado colaborativo es una técnica de recomendación basada en las interacciones entre usuarios y productos. Su idea principal es que usuarios con comportamientos similares pueden compartir intereses similares. Por ejemplo, si dos usuarios han interactuado con productos parecidos, es posible recomendar a uno de ellos productos que el otro ya compró o consultó.

Este enfoque es útil cuando se dispone de una gran cantidad de datos históricos de interacción. En el caso del dataset RetailRocket, las interacciones registradas permiten construir

una matriz usuario-producto que puede ser utilizada para identificar patrones de comportamiento y generar recomendaciones personalizadas.

III-D. Factorización matricial y ALS

La factorización matricial es una técnica utilizada para representar usuarios y productos en un espacio de menor dimensión. En lugar de trabajar directamente con una matriz grande y dispersa de interacciones, el modelo aprende vectores latentes para cada usuario y cada producto. La similitud entre estos vectores permite estimar qué tan relevante puede ser un producto para un usuario.

Uno de los métodos utilizados para este tipo de problema es ALS, conocido como Alternating Least Squares. Este algoritmo ajusta de forma alternada los factores latentes de usuarios y productos. En proyectos de recomendación con retroalimentación implícita, ALS puede utilizarse como modelo base, ya que permite construir recomendaciones de manera relativamente eficiente y comparar sus resultados contra modelos más avanzados.

III-E. Modelos basados en embeddings

Los modelos basados en embeddings representan usuarios y productos mediante vectores aprendidos automáticamente durante el entrenamiento. Estos vectores buscan capturar relaciones ocultas entre los elementos del sistema. Si un usuario y un producto tienen embeddings cercanos, se interpreta que existe una posible afinidad entre ambos.

Una arquitectura útil en sistemas de recomendación es el modelo Two-Tower. En este enfoque, una parte del modelo aprende la representación del usuario y otra parte aprende la representación del producto. Luego, ambas representaciones se comparan para estimar la probabilidad de interacción. Este diseño puede ser conveniente para sistemas de recomendación a gran escala, ya que permite calcular previamente representaciones de productos y acelerar la inferencia.

III-F. Computación paralela y distribuida

La computación paralela consiste en dividir una tarea en varias partes que pueden ejecutarse simultáneamente. Esto permite aprovechar múltiples núcleos de CPU o unidades de procesamiento para reducir el tiempo total de ejecución. En este proyecto, el paralelismo resulta importante porque el dataset contiene millones de eventos de interacción, lo que puede volver lento el procesamiento secuencial tradicional.

La computación distribuida, por su parte, permite repartir el trabajo entre varios procesos, workers o nodos. Esta estrategia es útil cuando el volumen de datos crece o cuando se busca simular escenarios de alta demanda, como picos de tráfico en eventos comerciales masivos. En el sistema propuesto, el procesamiento distribuido se considera principalmente para particionar datos, ejecutar transformaciones y evaluar la escalabilidad del pipeline.

III-G. Aceleración por GPU

La aceleración por GPU se utiliza cuando una tarea puede dividirse en muchas operaciones pequeñas que se ejecutan en paralelo. A diferencia de una CPU, que posee pocos núcleos más complejos, una GPU cuenta con una gran cantidad de unidades de procesamiento diseñadas para ejecutar operaciones masivamente paralelas.

En este proyecto, la GPU puede utilizarse en tareas como clustering de usuarios, cálculo de similitudes y entrenamiento de modelos basados en embeddings. Herramientas como RAPIDS cuML y PyTorch permiten aprovechar este tipo de hardware para reducir tiempos de entrenamiento o análisis en comparación con enfoques ejecutados únicamente en CPU.

III-H. Herramientas de procesamiento de datos

Polars es una biblioteca para procesamiento de datos tabulares que aprovecha el paralelismo multinúcleo y optimización de consultas. Su uso se justifica en etapas como limpieza, filtrado, agregaciones y joins, donde se requiere procesar grandes volúmenes de datos de forma eficiente.

Dask permite trabajar con datos divididos en particiones y distribuir tareas entre varios workers. Esta herramienta es útil para simular el procesamiento distribuido y evaluar la escalabilidad del pipeline. Por otro lado, PyArrow y el formato Parquet permiten almacenar datos en formato columnar, facilitando la interoperabilidad entre distintas herramientas y reduciendo costos de entrada/salida.

III-I. Métricas de calidad del sistema de recomendación

La evaluación de un sistema de recomendación requiere métricas orientadas al ranking. No basta con saber si el sistema acierta o falla, sino que también importa el orden en que aparecen los productos recomendados.

Entre las métricas relevantes se encuentran Hit Rate@K, MAP@K y NDCG@K. Hit Rate@K indica si al menos un producto relevante aparece dentro de las primeras K recomendaciones. MAP@K evalúa la precisión promedio considerando el orden de los elementos recomendados. NDCG@K asigna mayor importancia a los aciertos ubicados en las primeras posiciones del ranking.

III-J. Métricas de rendimiento computacional

Debido a que el proyecto pertenece al área de computación paralela y distribuida, también es necesario evaluar el rendimiento computacional. Las métricas principales son tiempo de ejecución, speedup y eficiencia.

El tiempo de ejecución mide cuánto tarda una etapa del pipeline en completarse. El speedup compara el tiempo de una versión secuencial contra una versión paralela o distribuida. La eficiencia relaciona el speedup obtenido con la cantidad de recursos utilizados, permitiendo analizar si el uso de más núcleos, workers o GPU realmente produce una mejora proporcional.

Estas métricas permiten justificar técnicamente el uso de herramientas paralelas y distribuidas, demostrando si la solución propuesta mejora el procesamiento frente a un enfoque tradicional.

IV. METODOLOGÍA

La metodología del proyecto se desarrollará mediante un flujo de trabajo dividido en etapas, con el fin de construir, evaluar e integrar un sistema de recomendación para comercio electrónico basado en procesamiento paralelo y distribuido. Cada etapa cumple una función específica dentro del pipeline general, desde la adquisición de datos hasta la visualización de resultados y la evaluación del rendimiento computacional.

IV-A. Adquisición y revisión inicial de datos

La primera etapa consiste en utilizar el dataset RetailRocket E-commerce Dataset, el cual contiene aproximadamente 2.7 millones de eventos de interacción entre usuarios y productos. Estos eventos incluyen visualizaciones, adiciones al carrito y transacciones, además de información complementaria sobre categorías y propiedades de productos.

En esta fase se revisará la estructura de los archivos disponibles, los tipos de datos, la cantidad de registros, la relación entre las tablas y la calidad general de la información. También se identificarán posibles valores faltantes, duplicados o inconsistencias que deban ser tratados antes del análisis y modelado.

IV-B. Preprocesamiento de datos

Después de la revisión inicial, se realizará la limpieza y transformación del conjunto de datos. Esta etapa incluirá la normalización de tipos de datos, eliminación de registros inconsistentes, integración de fuentes relacionadas y creación de variables derivadas a partir de las interacciones usuario-producto.

Para mejorar el rendimiento del procesamiento, se utilizarán herramientas como Polars y Dask. Polars se empleará para operaciones tabulares eficientes, como filtrados, agregaciones y joins. Dask se utilizará en aquellas tareas donde sea necesario particionar el dataset y distribuir el procesamiento entre varios workers. La salida del preprocesamiento se almacenará en formato Parquet, con el fin de facilitar su reutilización en las siguientes etapas.

IV-C. Análisis exploratorio de datos

Con los datos limpios, se realizará un análisis exploratorio orientado a comprender el comportamiento de los usuarios y productos dentro del sistema. Se estudiará la distribución de eventos, la frecuencia de interacción por usuario, la popularidad de productos y la proporción entre visualizaciones, adiciones al carrito y compras.

También se analizarán patrones relevantes que puedan apoyar el diseño del sistema de recomendación, como usuarios con alto nivel de actividad, productos con mayor frecuencia de interacción y posibles diferencias entre tipos de eventos. Los resultados de esta etapa permitirán seleccionar variables útiles para el modelado posterior.

IV-D. Segmentación de usuarios

Como parte del análisis del comportamiento, se aplicarán técnicas de clustering para identificar grupos de usuarios con patrones similares de interacción. Para ello se construirán variables agregadas por usuario, tales como cantidad total de eventos, diversidad de productos consultados, número de compras y frecuencia de actividad.

Se evaluará una implementación en CPU mediante Scikit-Learn y, si el entorno de ejecución lo permite, una versión acelerada por GPU mediante RAPIDS cuML. Esta comparación permitirá observar si la aceleración por GPU aporta una mejora significativa en tiempo de ejecución para esta etapa.

IV-E. Construcción del sistema de recomendación

El sistema de recomendación se desarrollará en dos niveles. Primero se implementará un modelo base de filtrado colaborativo, que servirá como referencia inicial para medir la calidad de las recomendaciones. Posteriormente, se evaluará un modelo más avanzado basado en embeddings, implementado con PyTorch, con posibilidad de utilizar aceleración por GPU durante el entrenamiento.

Ambos modelos utilizarán las interacciones históricas entre usuarios y productos para generar recomendaciones personalizadas. La salida esperada será una lista top-N de productos recomendados para cada usuario.

IV-F. Evaluación de recomendaciones

La calidad del sistema será evaluada mediante métricas orientadas al ranking, tales como Hit Rate@K, MAP@K y NDCG@K. Estas métricas permitirán analizar si los productos relevantes aparecen dentro de las primeras posiciones recomendadas y si el orden de las recomendaciones es adecuado.

Además, se revisarán casos donde el sistema presente menor desempeño, especialmente usuarios con pocas interacciones o productos con baja frecuencia. Este análisis permitirá identificar limitaciones del modelo y posibles mejoras para futuras iteraciones.

IV-G. Evaluación del rendimiento computacional

De forma transversal, se medirán los tiempos de ejecución de las principales etapas del pipeline: carga de datos, limpieza, transformaciones, agregaciones, clustering, entrenamiento e inferencia. Se compararán versiones base contra versiones paralelas o distribuidas cuando sea posible.

Las métricas consideradas serán tiempo de ejecución, speedup, eficiencia, uso de memoria y latencia de inferencia. También se utilizarán herramientas de profiling para identificar cuellos de botella y determinar qué partes del sistema se benefician más del paralelismo multinúcleo, procesamiento distribuido o aceleración por GPU.

IV-H. Visualización e integración final

Finalmente, se desarrollará un dashboard interactivo utilizando herramientas como Dash y Plotly. Este dashboard permitirá visualizar estadísticas del dataset, segmentos de

usuarios, recomendaciones generadas, métricas de calidad del modelo y resultados de rendimiento.

La integración final del proyecto se organizará en un repositorio GitLab, incluyendo código fuente, notebooks, resultados experimentales, instrucciones de ejecución y documentación necesaria para reproducir el trabajo.

V. PROBLEMA

V-A. Contexto

El comercio electrónico genera diariamente millones de interacciones entre usuarios y productos, tales como visualizaciones, adiciones al carrito y compras. Estos datos son la materia prima de los sistemas de recomendación, componentes que hoy resultan indispensables para mejorar la experiencia del cliente, aumentar la conversión y fortalecer la fidelización.

V-B. Problema central

El reto no es solo generar recomendaciones precisas, sino generarlas rápido y a gran escala. Los enfoques secuenciales tradicionales no logran procesar el volumen de datos que producen plataformas reales en tiempos aceptables, y esta limitación se agrava durante picos de tráfico, como un Black Friday, donde el número de interacciones puede multiplicarse en cuestión de minutos.

Un sistema que funciona bien en condiciones normales puede volverse inutilizable justo cuando más se necesita: en el momento de mayor demanda del negocio. Esto plantea una pregunta concreta: ¿cómo diseñar un sistema de recomendación que mantenga baja latencia y alta calidad de predicción incluso cuando el volumen de datos y usuarios crece de forma masiva y repentina?

V-C. Motivación

Responder a esta pregunta exige ir más allá del algoritmo de recomendación en sí y abordar el problema desde la arquitectura de cómputo. Es necesario determinar qué tareas conviene paralelizar en CPU multinúcleo, cuáles se benefician de la aceleración GPU y cuándo es necesario distribuir la carga entre varios nodos.

Este proyecto usa el dataset RetailRocket, con aproximadamente 2.7 millones de eventos, como caso de estudio realista para explorar estas decisiones. Esto permitirá evidenciar de forma práctica y medible, mediante métricas como speedup y eficiencia, cómo la computación paralela y distribuida puede resolver un problema que afecta directamente a millones de usuarios en su vida cotidiana.

VI. OBJETIVOS ESPECÍFICOS

1. Analizar y preparar los datos de interacción usuario-producto mediante el estudio estadístico de su distribución, la identificación de correlaciones relevantes en el comportamiento de los usuarios y la selección de variables representativas para el modelo de recomendación, aprovechando procesamiento paralelo para acelerar esta etapa sobre el volumen completo del dataset.

2. Diseñar e implementar un pipeline distribuido de recomendación que incluya la segmentación de usuarios mediante algoritmos de clustering y el cálculo de similitud o predicción acelerado por GPU, garantizando que el sistema pueda generar recomendaciones en tiempo real a partir de grandes volúmenes de datos.
3. Evaluar el rendimiento y la escalabilidad del sistema en escenarios que simulan condiciones reales de comercio electrónico, incluyendo picos de tráfico tipo Black Friday, midiendo métricas como speedup, eficiencia y latencia para comparar el impacto de distintas estrategias de paralelización frente a un enfoque secuencial.

VII. JUSTIFICACIÓN TÉCNICA

Las herramientas del proyecto se seleccionaron bajo un criterio principal: cada una debe permitir demostrar de forma medible un tipo de paralelismo sobre el dataset RetailRocket. Además, se priorizaron librerías con una ruta de trabajo clara entre CPU y GPU, debido a que el equipo puede trabajar en máquinas heterogéneas, algunas con GPU local y otras sin este recurso.

VII-A. Ingesta y preprocesamiento de datos

Se descartó Pandas como herramienta principal debido a que funciona mejor como referencia base o baseline secuencial para este proyecto. Aunque es una herramienta ampliamente utilizada, su enfoque puede convertirse en un cuello de botella cuando el volumen de datos crece. En su lugar, se seleccionó Polars como motor principal de limpieza y transformación, y Dask para las operaciones que requieren particionar el dataset entre varios workers.

- **Pandas:** se utilizará únicamente como referencia secuencial para comparar rendimiento.
- **Polars:** se empleará para limpieza, joins y agregaciones, aprovechando paralelismo multinúcleo y evaluación diferida.
- **Dask:** se utilizará para particionar el dataset y simular procesamiento distribuido mediante workers.
- **cuDF:** se evaluará en pruebas específicas donde sea posible aprovechar aceleración GPU.

Polars se justifica frente a Dask para las transformaciones tabulares simples porque su motor de ejecución aprovecha paralelismo multinúcleo sin la sobrecarga de un scheduler distribuido. Dask se reserva para etapas donde el dataset se particiona explícitamente y se necesita simular procesamiento distribuido. La salida de esta etapa se estandarizará en formato Parquet mediante PyArrow, ya que este formato reduce costos de entrada y salida y facilita la interoperabilidad entre herramientas.

VII-B. Análisis exploratorio y clustering de usuarios

Para el análisis exploratorio y la segmentación de usuarios se compararán Scikit-Learn en CPU y RAPIDS cuML en GPU. Scikit-Learn se utilizará como baseline, ya que permite ejecutar modelos clásicos y clustering en CPU. RAPIDS cuML será evaluado como alternativa acelerada por GPU para medir

posibles mejoras en tareas como K-Means o DBSCAN sobre grandes volúmenes de datos.

Esta comparación es importante porque permite medir el efecto real de paralelizar el análisis exploratorio. Sin una versión base ejecutada en CPU, no sería posible calcular speedup ni justificar técnicamente el uso de GPU.

VII-C. Modelado y aceleración por GPU

El núcleo del sistema de recomendación combina un modelo baseline y un modelo acelerado por GPU. Como baseline se propone utilizar Implicit ALS, debido a que permite trabajar con filtrado colaborativo sobre datos de retroalimentación implícita. Este modelo es útil porque no requiere GPU, es relativamente rápido de entrenar y permite obtener una referencia inicial de calidad mediante métricas como NDCG@K, MAP@K y Hit Rate@K.

Para el modelo avanzado se utilizará PyTorch, con una arquitectura basada en embeddings o Two-Tower. Esta herramienta se selecciona por su flexibilidad, soporte para GPU y facilidad de integración con etapas posteriores de inferencia. RAPIDS cuML también se evaluará como alternativa para acelerar cálculos de similitud o vecinos más cercanos, tareas necesarias para generar recomendaciones en tiempo reducido.

VII-D. Simulación de picos de tráfico y escalabilidad

Para evaluar escenarios similares a un aumento repentino de demanda, como Black Friday, Dask se utilizará para simular incremento del volumen de eventos y repartir la carga entre varios workers. Esto permitirá analizar tanto escalabilidad fuerte como escalabilidad débil.

El profiling de cada etapa se realizará con herramientas como cProfile y line_profiler, debido a que permiten identificar cuellos de botella a nivel de función sin añadir dependencias pesadas. Cuando se utilice GPU, se considerará el monitoreo de recursos mediante herramientas especializadas para verificar si la aceleración se está aprovechando de forma adecuada.

VIII. DISEÑO DE LA ARQUITECTURA GENERAL

El sistema se organiza como un pipeline modular. Primero se realiza la adquisición y gestión de datos. Luego, los datos pasan por un proceso de preprocesamiento paralelo. Posteriormente, se ejecuta el análisis exploratorio, la segmentación y el modelado de recomendación. Finalmente, se evalúan los resultados y se presentan mediante un dashboard interactivo.

VIII-A. Etapas del pipeline

1. **Adquisición y gestión de datos:** el pipeline inicia con la ingesta del dataset RetailRocket E-commerce Dataset, el cual contiene eventos de interacción usuario-producto, además de catálogos de categorías y propiedades de productos.
2. **Preprocesamiento paralelo:** los datos crudos se limpian, integran y transforman utilizando Polars y Dask. La salida de esta etapa se estandariza en formato Parquet particionado.

3. **Análisis exploratorio y estadístico:** sobre los datos preprocesados se ejecuta el análisis descriptivo, identificación de correlaciones y segmentación de usuarios mediante clustering.
4. **Modelado de Machine Learning:** con las variables seleccionadas se entrena primero un modelo baseline de filtrado colaborativo y luego un modelo más avanzado basado en PyTorch.
5. **Postprocesamiento y análisis de resultados:** los resultados de ambos modelos se evalúan con métricas estándar de sistemas de recomendación, tales como NDCG@K, MAP@K y Hit Rate@K.
6. **Visualización y difusión de resultados:** un dashboard interactivo construido con Dash y Plotly expone las recomendaciones generadas, los resultados del análisis exploratorio y las métricas de rendimiento.

VIII-B. Módulo transversal de rendimiento

El análisis de rendimiento se considera un módulo transversal, ya que no ocupa un lugar fijo en la secuencia del pipeline. Este módulo mide el tiempo de ejecución, speedup y eficiencia de las principales etapas, comparando una versión base contra versiones paralelas o distribuidas. Además, identifica cuellos de botella mediante herramientas de profiling.

IX. PLAN DE TRABAJO

El proyecto se organiza en cuatro entregas principales, alineadas con el avance esperado del curso. Cada entrega tiene un propósito específico y permite construir progresivamente la solución final.

IX-A. Entrega 1: Fundamentos y diseño del proyecto

La primera entrega se enfoca en definir el problema, revisar literatura relacionada, seleccionar el dataset, establecer los objetivos específicos, justificar las herramientas seleccionadas y proponer la arquitectura general del sistema. También se consolida el documento inicial en formato IEEE.

Las actividades principales de esta entrega son:

- Definición del problema y selección final del dataset RetailRocket.
- Redacción de la introducción, problema y objetivos específicos.
- Investigación y justificación técnica de herramientas.
- Definición de la arquitectura general del pipeline.
- Elaboración del cronograma y revisión final del documento.

IX-B. Entrega 2: Implementación inicial y validación parcial

La segunda entrega se centra en demostrar avance funcional. Se espera contar con ingesta de datos, preprocesamiento paralelo o distribuido, análisis exploratorio, clustering inicial y pruebas preliminares de aceleración.

Las actividades principales son:

- Ingesta de datos y preprocesamiento con Polars y Dask.
- Análisis exploratorio sobre datos reales.
- Segmentación inicial de usuarios mediante clustering.

- Primeras pruebas de aceleración por GPU.
- Validación preliminar de resultados y documentación de obstáculos.

IX-C. Entrega 3: Proyecto final completo

La tercera entrega corresponde al sistema integrado. En esta fase se espera contar con el modelo de recomendación, análisis de rendimiento, simulación de picos de tráfico, postprocesamiento y dashboard interactivo.

Las actividades principales son:

- Modelado del sistema de recomendación con Scikit-Learn, cuML o PyTorch.
- Escalamiento distribuido y simulación de picos de tráfico.
- Evaluación de métricas de recomendación.
- Medición de speedup, eficiencia, uso de GPU y memoria.
- Desarrollo del dashboard interactivo.
- Revisión final del código, README y reproducibilidad en GitLab.

IX-D. Entrega 4: Presentación

La cuarta entrega corresponde a la presentación final del proyecto. En esta etapa se prepara la demostración del dashboard, se distribuyen los tiempos de exposición y se presentan los resultados obtenidos.

Las actividades principales son:

- Preparación de la presentación y demostración del dashboard.
- Ensayo general y distribución de tiempos entre integrantes.
- Presentación final del proyecto.

X. CONCLUSIONES PRELIMINARES

El proyecto propuesto aborda un problema realista de comercio electrónico donde el volumen de datos y la necesidad de baja latencia justifican el uso de técnicas de computación paralela y distribuida. La selección del dataset RetailRocket permite trabajar con millones de interacciones reales, lo que ofrece un escenario adecuado para evaluar herramientas como Polars, Dask, RAPIDS cuML y PyTorch.

La propuesta no se limita a construir un modelo de recomendación, sino que incorpora análisis de rendimiento computacional como parte central del diseño. Esto permitirá comparar enfoques secuenciales, paralelos, distribuidos y acelerados por GPU, generando evidencia sobre el impacto real de cada estrategia en el procesamiento de datos y la generación de recomendaciones.

Además, la inclusión de un dashboard interactivo permitirá comunicar los resultados de forma clara y facilitar la exploración de recomendaciones, métricas del modelo y resultados de rendimiento. De esta manera, el proyecto integra ciencia de datos, Machine Learning y computación paralela en una solución aplicada a un escenario de alto impacto.

REFERENCIAS

- [1] F. Ricci, L. Rokach, and B. Shapira, *Recommender Systems Handbook*. Springer, 2015.
- [2] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.
- [3] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative filtering for implicit feedback datasets," in *Proc. IEEE International Conference on Data Mining*, 2008, pp. 263–272.
- [4] P. Covington, J. Adams, and E. Sargin, "Deep neural networks for YouTube recommendations," in *Proc. ACM Conference on Recommender Systems*, 2016, pp. 191–198.
- [5] Dask Development Team, "Dask: Library for dynamic task scheduling," 2026. [Online]. Available: <https://www.dask.org/>
- [6] Polars Developers, "Polars: DataFrames for the new era," 2026. [Online]. Available: <https://pola.rs/>
- [7] NVIDIA RAPIDS, "RAPIDS: Open GPU Data Science," 2026. [Online]. Available: <https://rapids.ai/>
- [8] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.